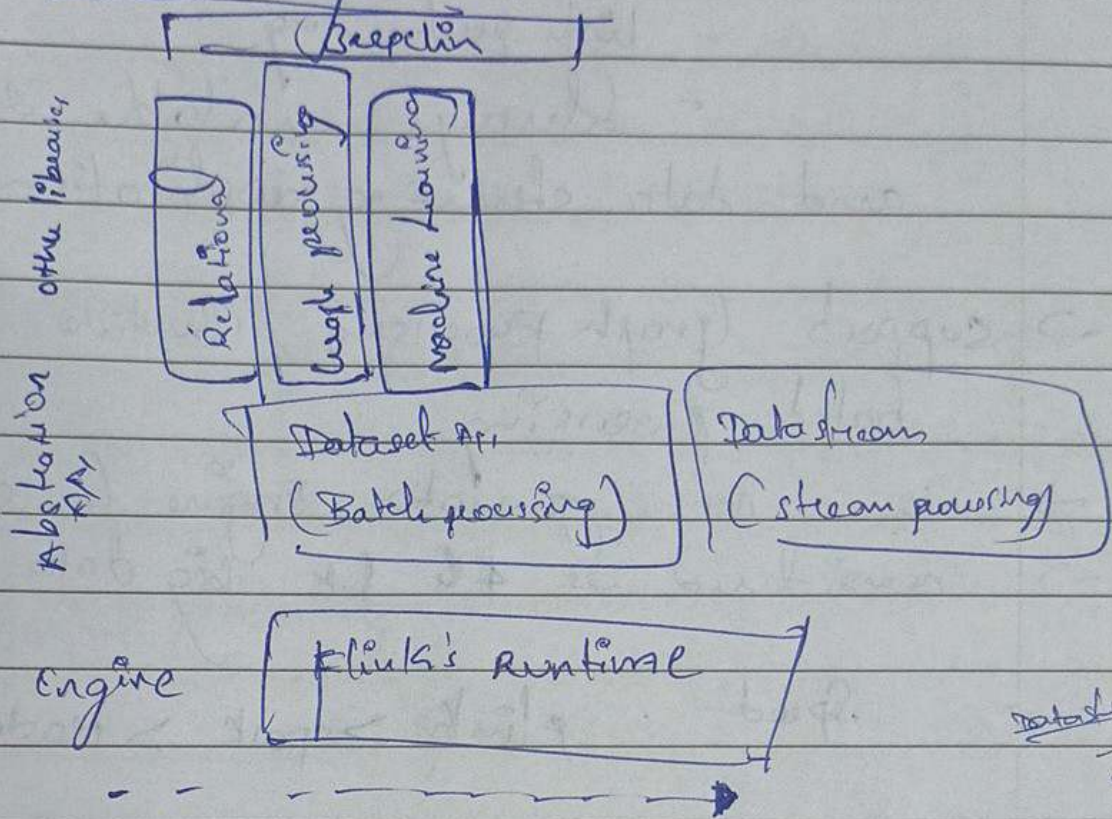


Flink ecosystem:



for each of task flink has built in libraries.

Dataset API
 ↳ used in
 java, scala,
 python

Relational - Table API
 (join, groupby, ...)

(Table API can be used in both batch as streaming programs)

Graph - Gelly API

~~ML~~
 ML - PLink ML

↳ Zeppelin - visualization tool - allow to view each step of processing of query

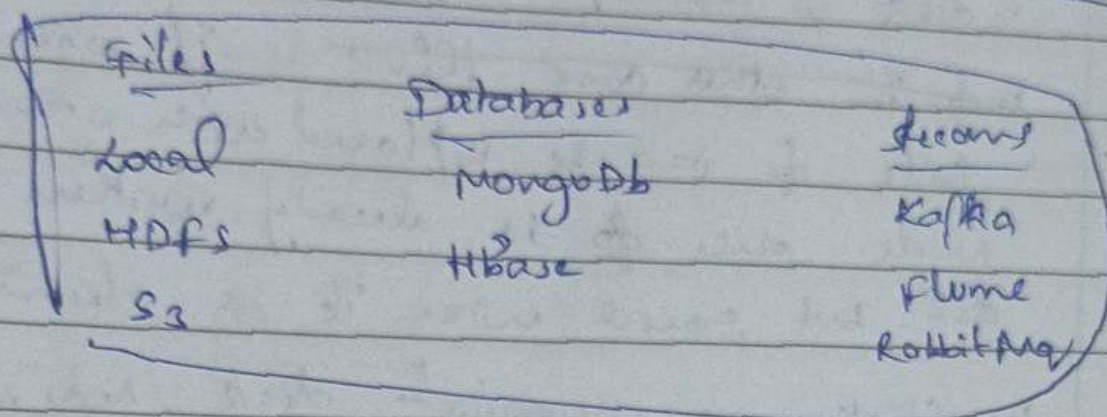
Display

Local JVM

cluster
Standalone/ yarn

cloud
GCE/EC2

Storage



⇒

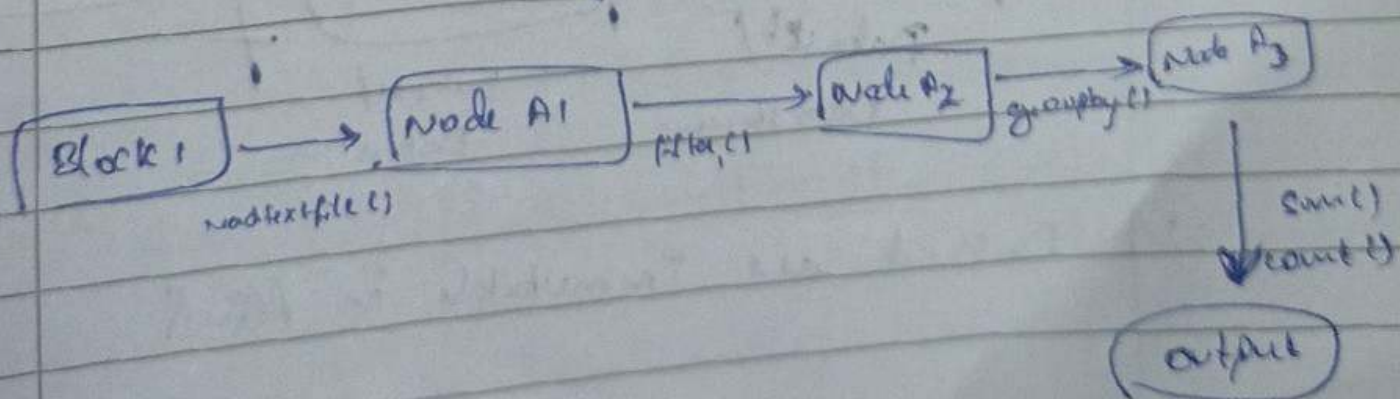
`DataSet < String > text = env.readTextFile(params.get("input"))`

`DataSet < String > filtered = text.filter(new FilterFunction < String > ()`

`public boolean filter(String value)`

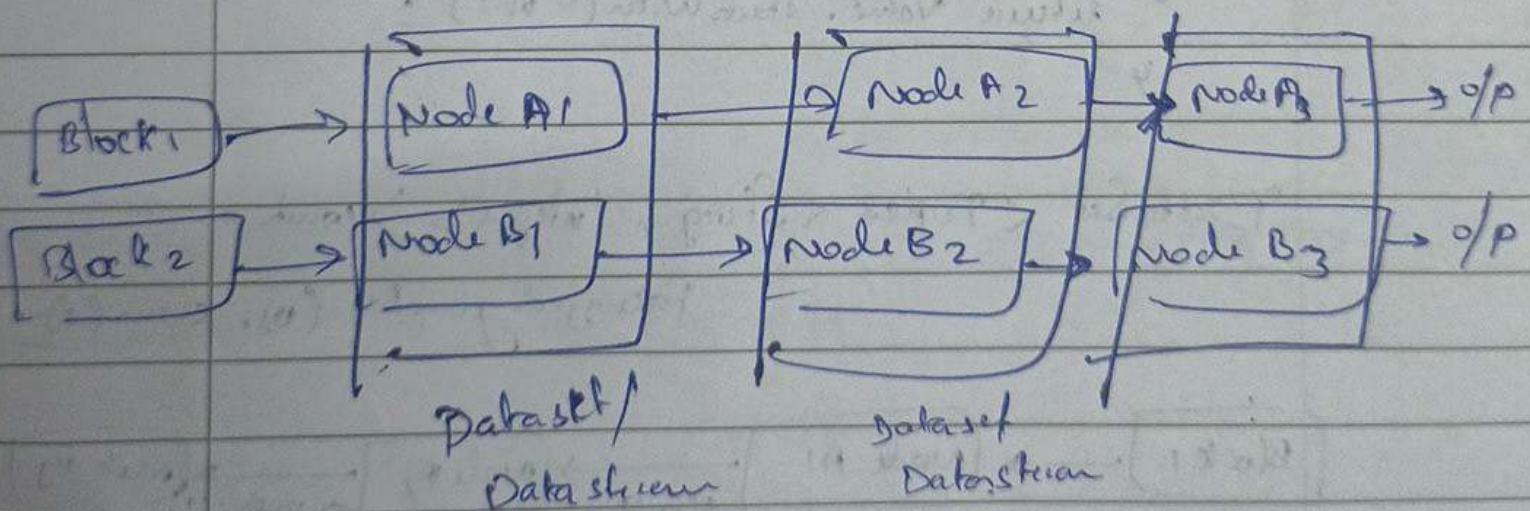
`{ return value.startsWith("n"); }`

`DataSet < Tuple2 < String, Integer > > count =
filtered.groupBy (0).sum(1);`



→ In Hadoop intermediate operation are not built in. It use only map-reduce & each each operation are stored in disk, so if one fails it affects other but in other case flink if one node fails it can be replaced with other active node since op is already written to the disk but speed wise it is slow

→ In flink, no intermediate disk, node 2 will directly read from node 1 memory. if a node fails then next node will perform, but how it knows what operation node 1 is performing? Flink Run knew that Node A1 was performing which operations on which dataset.



→ Dataset are immutable in flink

only dataset is created we cannot change elements inside it

→ we cannot perform different operation on dataset whole dataset or not for half of dataset
(any operation in flink creates new dataset)

⇒ Code structure

we can provide local or remote environment

↓
call execution
in current JVM

↘
execution on
remote cluster

⇒

- readTextFile (path) - Reads file line wise and returns them as dataset of strings.
- readCsvFile (path) - takes a csv file as input & returns dataset of tuples.
- readFileOfPrimitives (path, class) - reads each line of file as line of class not mentioned in argument (if integer, reads integer)
- readFileOfPrimitives (path, delimiter, class) - reads file with delimiter (common separated)
- readHadoopFile (FileInputFormat, Key, value, path) - Reads Hadoop file
- readSequenceFile (Key, value, path) - Reads sequence file

Ex 1 -

DataSet <String> text = env.readTextFile (params.get("input"))

DataSet <String> filtered = text.filter(new FilterFunction <String>()

```
{
    public filter (String value)
    {
        return value.startsWith("N");
    }
});
```

// Returns a tuple of (name, 1)

DataSet < Tuple2 <String, Integer> > tokenized =
filtered.map(new Tokenizer());

// group by tuple field "0" or sum up tuple field "1"

DataSet < Tuple2 <String, Integer> > count = tokenized.
groupBy(0).sum()

if (params.has("output"))

```
{
    count.writeAsText(params.get("output"),
        "\n", "\n");
    env.execute("wordcount example");
}
```


flatMap example

```
Dataset<String> text = env.readTextFile(  
    Params.get("input")
```

```
)  
Dataset<Tuple2<String, Integer>> tokenized  
    = text.flatMap(new Tokenizer())
```

P.S. final class Tokenizer implements flatMapFunction
 <String, Tuple2<String, Integer>>

```
{  
    P.V. flatMap (String value, Collector<Tuple2<String, Integer>>  
        out)
```

```
{  
    String[] tokens = value.split(" ")
```

```
    for (String token : tokens)
```

```
    {  
        if (token.length() > 0)
```

```
        {  
            out.collect(new Tuple2<String,  
                Integer
```

```
                (token, 1))
```


tuples in this
 start with
 zero reference
 - / -

Inner Join

DataSet < Tuple2 < Integer, String > > personSet = env.readTextFile
 (params.get("input1"))

• map (new MapFunction < String, Tuple2 < Integer, String > > {

{
 p. tuple2 < Integer, String > map (string value)

{
 String[] words = value.split(",");

return new Tuple2 < Integer, String >

(Integer.parseInt(words[0]),
 words[1]);

}

});

// similarly input 2

DataSet < Tuple3 < Integer, String, String > > joined =

personSet.join(locationSet).where (o).equalTo(o)

• with (new JoinFunction < Tuple2 < Integer, String >, person

Tuple2 < Integer, String > location)

{

return new Tuple3<Integer, String, String>
(person.f0, person.f1, location.f1);

3)
3)

joined.writeAsCSV(params.get("output"), "a", "a");
env.execute("join example");

3

Left join

if (location == null)

{ return new Tuple3<Integer, String, String>
(person.f0, person.f1, "null");
}

Join Hint

- OPTIMIZER - CHOOSES
- BROADCAST - HASH - FIRST - second

[if we know one second dataset is small, then spark will broadcast to all nodes in then do join]

Left outer join

= matching

records from

both +

non matching from left

Right outer join

= matching

records

from both

+ non matching from right

Full outer join

= matching and

from both

+ non matching from both

3) REPARTITION - HASH - FIRST

(builds hash table from first input)
- useful when both inputs are large when first input is smaller than second

4) REPARTITION - HASH - SECOND

- when second input is smaller

5) REPARTITION - SORT - MERGE

Join is performed on sorted input
useful when we have one or both sorted inputs.

DATASTREAM operation

Data sources for datastream API

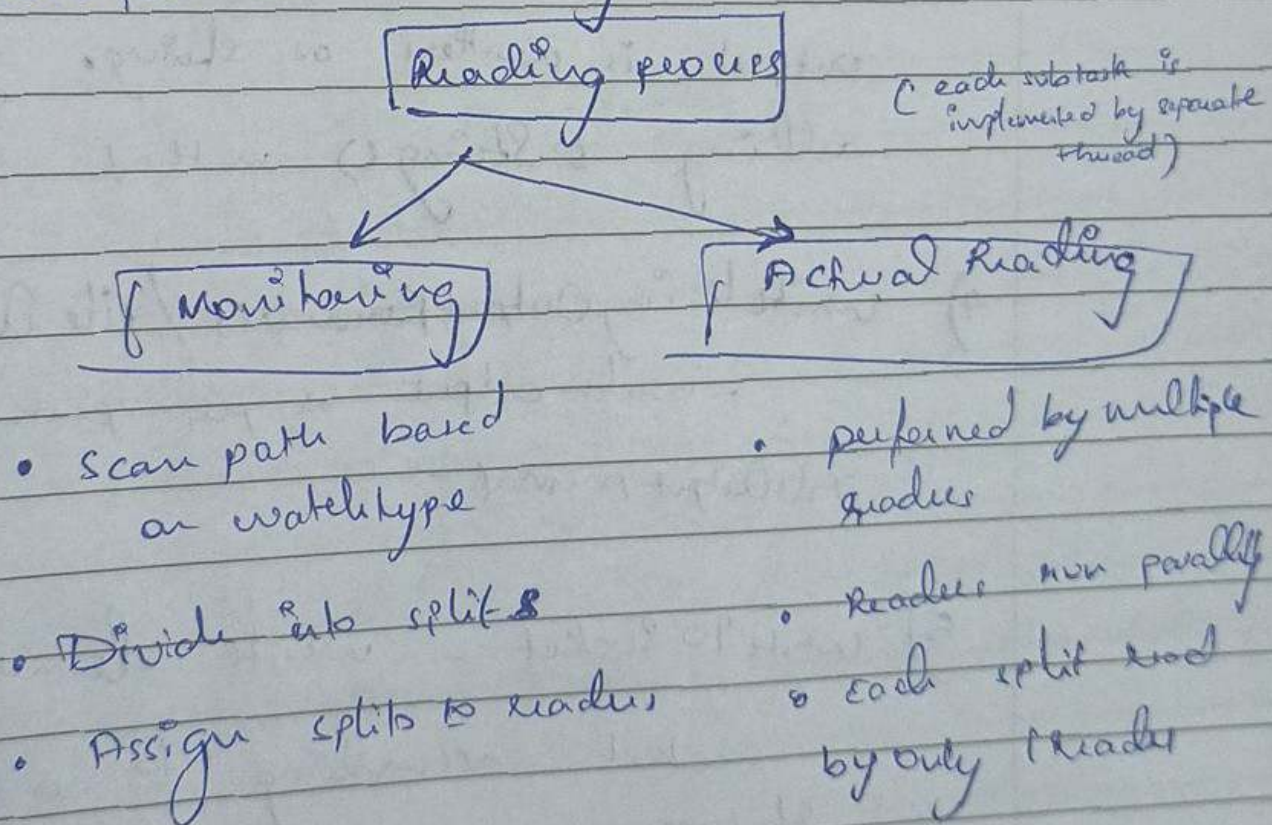
- readTextFile (path) - Reads line by line and returns them as string
- readFile (fileInputFormat, path) - Read lines in format as mentioned in parameters
- readFile (fileInputFormat, path, watchType, interval, pathFilter)
- socketTextStream - Reads data from socket
- addSource - to add a custom data source which of think ex:- Kafka, flume, Twitter API etc

Types of watch type :

1) File Processing Mode. PROCESS - CONTINUOUSLY
- checks the path for any given
Interval
(ex. 20 milliseconds)

2) File Processing Mode. PROCESS - ONCE
- scan the path once read the data
create a checkpoint or exists. It will
never visit the checkpoint again

* How does the Reading task?



Data sinks for datastream app

1) `writeAsText()` / Text Output Format -

writes output line wise, each line as a string

2) `writeAsCsv()` (path, lines delimiter, (fields delimiter))
on Csv Output Format - writes output as comma separated values.

3) `print()` - prints output to console.
output is written as string, by internally calling to `String()` method

4) `writeUsingOutputFormat()` / File Output Format
- write output as per provided File Output Format

5) `writeToSocket()` - write elements to a socket according to a serialization schema

6) `addSink()` - to add a custom data sink outside of flink ex: kafka, flume etc using connector.